Real-Time Systems – Design Project

Aim: To work through the process of designing a real-time system and demonstrate an implementation. Complexity of design will achieve a higher grade only if implemented in a sensible and justifiable manner – Your claims will need to back-up by evidence during demonstration.

You are to design a Traffic Light control system according to the requirements given in this document, then design and implement a "proof of concept" solution.

Project Overview

1. Each group must develop an initial design for the Traffic Light control system and submit an **initial design report**.
2. Develop a "proof-of-concept" implementation using the QNX Real-Time Operating System. Each group must then **present and demonstrate** their solution.
3. Submit the **final software (source code)** and **implementation note**.

Implementation: Implementation must be done using the QNX Real-Time operating system with software written in C or C++ and must use the QNX supported **interprocess communications (IPC) and synchronisation primitives**.

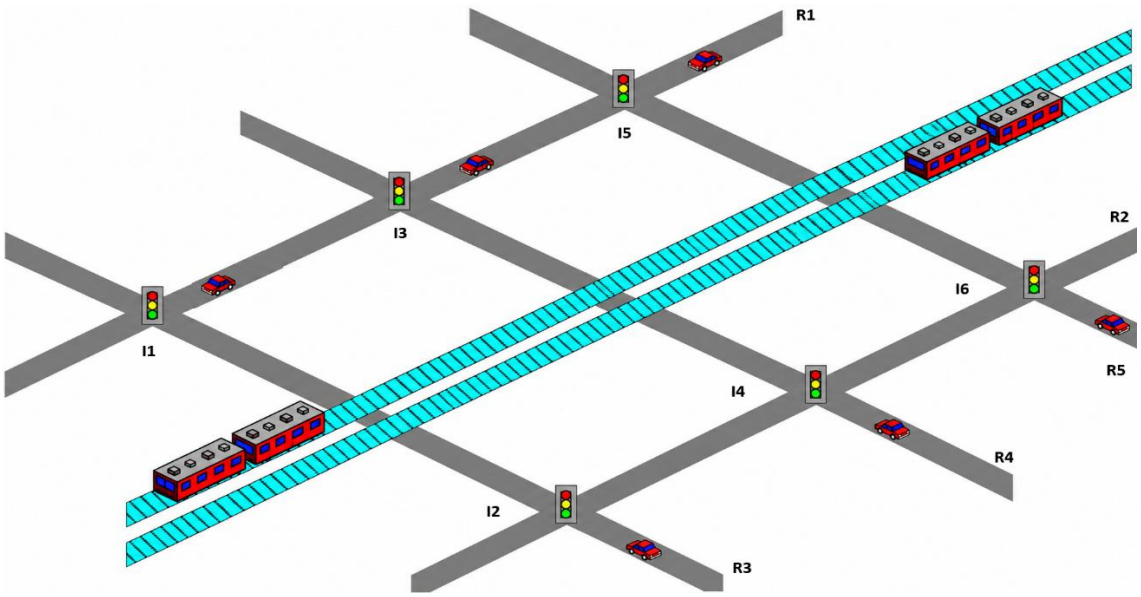
Project Requirements

Traffic Light Controller

Use the real-time systems design methodologies outlined in lectures to develop and design a real-time system for traffic light control according to the specifications that follow. You will be required to develop a functioning "proof-of-concept" implementation using multiple QNX nodes. Use Design Diagrams (including UML variants) to document your design.

Traffic Lights Control System Requirements

The system is to control six signalised intersections (I1-I6) and the associated railway-crossing protection for the double-track railway corridor, as shown in the following diagram. The minimum requirement to aim for a Pass grade is to implement a fully-functional 1/3 part of the map (2 intersections I1-I2 and the railway crossing system). The complete design should duplicate this part to create the full network with reasonable timing and coordination.



In the layout above, the road network comprises **five roads (R1-R5)** and six signalised i**ntersections (I1-I6)**. A double-track railway corridor runs diagonally through the network and interacts with multiple road approaches. All roads carry two-way traffic, have two lanes in each direction, may have turn lanes and pedestrian crossing lights at each intersection. Traffic demand and priority may differ between R1-R5; teams must state and justify the assumptions used for timing, coordination, and congestion control.

The railway crossing has **boom gates** and **red flashing lights** that operate whenever a train crosses the road. There are two train lines, one for trains travelling in each direction. The time between trains varies from frequent (as little as 2 minutes apart) at peak hour to infrequent (about every 20 minutes) at night (~10:00pm) - All night trains run on Friday and Saturday nights. The traffic light system can sense the state of the railway crossing (but has no control over it). The train should receive a red light if there are any issues with the boom gates – thus indicating an error which should be reported to the control room.

Each intersection/pedestrian crossing has a local controller for the lights at the intersection which operate continuously (except for hardware failure). Each local controller also communicates with a central controller (but should continue to operate if either communication link fails or the central controller goes offline).

The six **local intersection controllers** (I1-I6) operate independently and can receive commands from the central controller to select suitable light-sequence patterns for different times of day. Intersections must sense the state of the railway crossing while continuing to accommodate traffic across R1-R5. Pedestrian buttons and pedestrian signals should also be considered if you want to target for grades from a CR level or above.

There is a central control room with a central controller, which monitors the traffic lights and displays the status and light settings received from the individual intersections. Therefore, each local controller will send a status message to the central controller whenever any of the lights at that intersection change state, and act on any commands received from the central controller.

Note that the central controller does not directly control the individual lights at an intersection - that is always done by the local controller and must be maintained even in the event that the controller cannot communicate with an intersection of another controller. A full implementation would also have a train line controller that handles the communication link between the traffic central controller and the train lines.

Modelling your design on a real intersection:

Below is an example intersection in Melbourne which might help with breaking down the problem. You should at least attempt to base your design on a real intersection - this may help with you formulating your justifications and requirements. You can include tram line signaling if needed too.

By measuring the distance between the traffic light intersections and train crossing you can start to determine the timing requirements and traffic light sequences that will be needed in-order to clear the intersections and avoid traffic backing up. By making some realistic assumptions, the side roads can be eliminated from the design as they are not important. As an example, during peak time with trains approaching from both sides, it can take between 3 to 10 minutes to follow the path outlined by the red arrows if there are pedestrians trying to cross Glenroy Rd or Hartington St.

Image source: https://www.google.com.au/maps/@-37.705977,144.9167887,18z

Possible light sequence patterns for each intersection local controller (set by a command from the central controller) include:

- Fixed timing – commonly used during peak hours.
- Sensor driven (react to local car sensors & any pedestrian buttons) – commonly used in off-peak times.
- Advanced: updateable sequence structure or timing requests from central controller.

The central controller may send a command to change the sequence a few times a day (e.g. depending on the time of day).Operators in the central control room will occasionally send commands to the intersections, boom gate control and train approach signals system.

The central control room may also initiate override commands that are sent to the individual intersections to deal with exceptional situations (e.g. to provide a clear path for a visiting dignitary).

At all times the lights at each intersection must exhibit the "expected" behaviour of real traffic lights, however the timing between state changes may be speed up for practical and demonstration reasons.

These specifications do not describe every possible feature of the intersections, leaving some room for variations in solutions (e.g. pedestrian lights, right turn arrows etc at the intersections).

You are welcome to simulate sensor events in your implementation e.g. using key presses (this will require an additional process or thread to wait for the key presses). You are free to use console input, or GPIO developed on your own DE10 targets.

It is expected that your solution will have multiple QNX nodes with separate process on each node to handle the features such as pedestrian / sensor inputs and/or communication links, with another process used to display the traffic light states.

You are not required to have a graphical user interface; it is sufficient to display the state of each intersection by writing simple text messages to a terminal window. You can also make use of the persistent file system on each target under the `/fs` directory. You can store test vectors or debug information for offline analysis, or any other feature you think is suitable.

You are free to make any reasonable assumptions necessary to proceed with the design, however it is a good idea to justify your decision on a simplified real-life traffic light intersection.

Initial Design Report

The initial design report shows the steps you went through in the design process leading to your initial design. Expected design documentation includes:

- Your interpretation of the problem, and any assumptions you made about how the system will operate.
- A short discussion as to what grade level you intend to deliver your solution too. Note: you will not be held to this level, it will just be used to provide feedback at the appropriate level. You can change, drop, or add features to your final design/submission as you see fit to target a lower or higher level.
- A detailed diagram of the intersection layouts for your design indicating the various lights and sensors (which depend on the intersection features supported, such as right-turn arrows and pedestrian crossings).
- State Charts describing the lights behaviour at each intersection.
- Use Case diagram(s) and use case scenarios including relevant State Chart(s) and written specifications.
- Behavioural specifications such as sequence diagram(s) or collaboration diagrams may be included.
- High level implementation diagrams especially a task architecture diagram.
- Overview of messages sent between tasks (type, purpose and outline of message content).
- Add explanatory notes particularly to diagrams, to ensure that your design is fully documented and clearly explained.

Diagrams must be clearly labelled and numbered.

Report must have a Table of Contents, professional layout and proper page headers and footers. It should also include a contribution declaration from each team member and cover page.

Project Demonstration

Each team will be required to formally demonstrate their solution as part of the Implementation and Technical Demonstration assessment item. This is by far the most important item of the course. All team members are required to attend and present an aspect of the solution. Demonstrations will be scheduled by the lecturer. Each demonstration will be preformed in 2 parts and go for no more than 30 minutes in total. The first part will be a student driven demonstration of all features that will go for no more than 15 minutes. The remaining 15 minutes will be a Question and Answer (Q&A) session driven by the assessor. During the Q&A session you might be required to re- demonstrate a given aspect of your solution under a certain situation to prove your claims or discuss aspects of the code.

NOTE: All features and scenarios must be discussed and demonstrated in face to face mode. All team members must take part in the presenting/answering questions.

You will be assessed on your professionalism as well as your technical justifications and solution. You are expected to demonstrate where you used any interprocess communications (IPC) and synchronisation primitives, as well as how you tested your solution.

Implementation Note

In addition to the final software (source code), each group must provide a concise **Implementation Note** describing the design and operation of the system, including its major components, processes, communication mechanisms, and key design decisions. The note must also explain how the project source code is organised, clearly identifying the purpose of the main directories, files, and modules. Relevant screenshots should be included to demonstrate the implemented system, its principal features, and the outcomes of representative test scenarios. The Implementation Note should be clear, well structured, and provide sufficient information for the assessor to understand and evaluate the software implementation.